

Optimizing Linux scheduling based on global runqueue with SCX

1st Tang Qinan
School of Informatics
Xiamen University
Xiamen, China
tangqinan@stu.xmu.edu.cn

2nd Gao Xing
School of Informatics
Xiamen University
Xiamen, China
gaoxing@xmu.edu.cn

2nd Li Guilin
School of Informatics
Xiamen University
Xiamen, China
glli@xmu.edu.cn

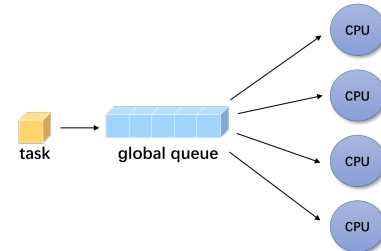
2nd Lin Juncong
School of Informatics
Xiamen University
Xiamen, China
jclin@xmu.edu.cn

Abstract—In Linux kernel version 6.6, the Earliest Eligible Virtual Deadline First (EEVDF) scheduler was introduced as the new default scheduler. However, due to its high computational complexity, it may not be suitable for all application scenarios. In cases where there are a large number of short-term tasks or frequent task communication, EEVDF can incur excessive context switch overhead and performance degradation. To address these issues, we propose a lightweight scheduling strategy named SRAND. Leveraging the programmable scheduling framework ‘sched_ext’ and employing BPF technology, we have implemented a strategy based on global and local run queues. This strategy utilizes a five-level BPF mapped queue to partition tasks with different virtual runtimes. Tasks with smaller virtual runtimes are placed into a FIFO-type global queue first, enabling priority scheduling for tasks with smaller virtual runtimes. Additionally, we monitor CPU idle states and allocate tasks in a timely manner, enhancing task responsiveness while reducing scheduling complexity. It is worth noting that our strategy integrates user-space scheduling policies into the kernel via an eBPF program loader, thus eliminating the need for kernel code modifications. By implementing the SRAND strategy, we have observed significant improvements compared to Linux’s default scheduling strategy EEVDF. Specifically, our proposed strategy averages an 11.83% reduction in process context switch time and an overall performance improvement of 7.02% in stress tests, while maintaining satisfactory load balancing.

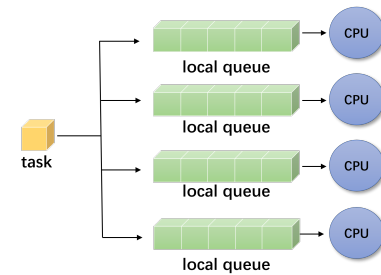
Keywords—custom scheduling policy, programmable scheduling framework, eBPF technology

I. INTRODUCTION

Scheduling is an essential operation in a computer operating system and occurs at multiple layers in the stack, from global load balancers to programmable network devices to the operating system in the end host [1]. The scheduler serves to multiplex CPU time among different tasks, such as processes and threads, involving two primary decisions: core selection and task selection. It determines which task to execute next, allocates appropriate time slices to each task, and designates the CPU core where the task should execute. By employing effective scheduling mechanisms, system resources can be optimized, facilitating concurrent execution of multiple processes.



(a) Centralized scheduling



(b) decentralized scheduling

Fig. 1: Two main scheduling methods in the development process of Linux

As shown in Figure 1, Linux scheduling policies have evolved from the initial $O(n)$ scheduler to the $O(1)$ scheduler. The introduction of the Completely Fair Scheduler [2] in kernel version 2.6.23 marked a significant milestone, as CFS served as the default scheduler in Linux for over 15 years. However, in November 2023, with the release of Linux kernel version 6.6, the Earliest Eligible Virtual Deadline First [3] [4] scheduler was introduced as the new default scheduler. This scheduling policy is essentially based on a general approach called Proportional Share Allocation (PSA) [5]. It utilizes per-cpu queues to arrange the execution order of tasks based on their virtual deadline priority, ensuring that tasks run without exceeding their deadlines.

This approach performs well on most hardware platforms and can handle the majority of workloads effectively. However, in desktop or server operating systems, when mul-

multiple users or applications run simultaneously, generating a large number of short-term tasks, the EEVDF scheduling policy needs to frequently calculate the lag value of each task for comparison, in order to determine the task with the shortest virtual deadline for scheduling. This frequent calculation and comparison incur significant overhead, leading to increased task context switch time and performance degradation.

As the generic scheduler EEVDF fails to meet the specific workload demands of handling a large number of short-term tasks in desktop systems, employing a custom scheduler for specific workloads in particular scenarios might prove advantageous. Previous practices have showcased numerous examples of such instances, such as the Steam Deck [6] gaming device, which typically runs Windows-based games on a Linux-based system, and eBay allowing its workloads on their mini data centers. However, implementing and maintaining custom schedulers is no easy feat. This is not only due to the challenges associated with replacing or customizing kernel scheduling policies but also because, in the long term, custom schedulers entail a significant maintenance burden. Nevertheless, the emergence of the `sched_ext` framework addresses these concerns. It is a secure and robust framework that facilitates rapid experimentation and deployment of various scheduling strategies.

Based on the above discussion, this paper proposes a new BPF scheduling policy called SRAND utilizing the programmable scheduling framework `sched_ext`. The algorithm employs a five-level BPF mapped queue to temporarily store tasks with different virtual runtimes. Subsequently, it traverses these five-level mapped queues, placing tasks with smaller virtual runtimes into a FIFO global queue. During scheduling, tasks are directly dequeued from the head of the global queue, ensuring that tasks with smaller virtual runtimes are executed first, thus reducing the complexity of task selection. In scenarios with a large number of short-term tasks, SRAND eliminates the need for extensive calculations and comparisons like EEVDF, instead directly dequeuing tasks from the queue's head, thereby simplifying task selection.

Additionally, we utilize a dynamic array to monitor whether each CPU core requires tasks and promptly allocate tasks to the local queues of idle CPUs. We then inject the SRAND scheduler into the kernel using an eBPF program loader to implement the new scheduling scheme. To our knowledge, this is one of the few scheduling policies implemented using a programmable scheduling framework.

We primarily benefit from the five-level eBPF mapped queue, FIFO global queue, and the `sched_ext` framework in the scheduler. The eBPF mapped queue facilitates the partitioning of tasks with different virtual runtimes, the global queue simplifies task management and allocation while ensuring tasks with smaller virtual runtimes are scheduled first, and the `sched_ext` framework streamlines the loading, unloading, running, and termination of BPF scheduling programs.

Specifically, this paper makes the following contributions:

- Proposing the utilization of a programmable scheduling framework `sched_ext` for implementing scheduling strategies.
- A five-level BPF mapping queue is proposed to temporarily store tasks in different virtual run time ranges.
- Implement the scheduling of global queues and local queues and conduct evaluation.

The rest of this paper is summarized as follows: Section 2 introduces related research on user-defined scheduling policies and eBPF. In Sect.3, we introduce the scheduling algorithm SRAND and its implementation proposed in this paper. Section 4 introduces the performance comparison experiments and results of the newly proposed scheduling algorithm and EEVDF. Section 5 concludes the paper.

II. RELATED WORK

A. user-defined scheduling strategies

D. R. Engler [7] is credited as the first researcher to conceptualize and propose user-defined kernel decisions. In his seminal work on the Exokernel, Engler introduced an innovative operating system architecture that utilizes a compact kernel to establish secure communication with untrusted users via a low-level interface. Within this framework, the library operating system exposes all hardware resources, leveraging this interface to implement system objects and policies—an early precursor to the eBPF framework.

Andrea C. Arpaci-Dusseau et al [8] contributed to the field by proposing the "infokernel" operating system architecture. This architecture enhances user control by exposing crucial internal state information, enabling users to manipulate and adapt the kernel's usage mechanisms. This approach fosters a more flexible and tailored operating system experience for users.

In a separate study, Joshua Fried et al [9] introduced the Caladan CPU scheduler. This novel scheduler comprises a centralized scheduler core and a kernel module, designed to be interference-aware. Additionally, Kostis Kaffes et al [1] presented the Syrup framework, which employs a matching function to identify corresponding scheduling strategies based on various inputs, including threads, network data packets, and network connections. This framework facilitates the implementation of custom scheduling strategies tailored to different loads.

Furthermore, Jack Tigar Humphries et al [10] proposed ghOSt, a platform that delegates scheduling policies from the kernel domain to user space, enhancing flexibility and customization options.

B. eBPF

eBPF represents a revolutionary technology enabling the safe and efficient extension of kernel functionality without necessitating alterations to the kernel source code or loading

of kernel modules. This advancement opens up the possibility of implementing user-defined schedulers.

Sebastiano Miano et al [11], introduced Polycube, leveraging eBPF to construct the data plane of network functions (NFs) and providing a streamlined development process not achievable within traditional kernels. Additionally, Matteo Bertrone et al [12], utilized eBPF to develop an iptables-like filter, bpf-iptables, capable of handling more intricate network requirements and diverse application scenarios. Yoann Ghigoff et al [13], utilized eBPF to introduce BMC, a novel Memcached kernel cache that demonstrates enhanced performance in processing requests prior to the execution of the network stack.

III. METHODOLOGY

The core idea behind BPF is to allow programs to be loaded from user space into kernel space at runtime. Compared to existing Linux kernel scheduling, using BPF for scheduling introduces significantly different scheduling behaviors. As shown in the Fig.2, our strategy involves temporarily storing non-kernel thread tasks in five eBPF map queues according to the virtual runtime, and designing the eBPF map queues as a first-in-first-out mode. Each queue stores tasks with different ranges of virtual runtimes, with lower-indexed queues containing tasks with fewer virtual runtimes. The eBPF mapped queues, from queue0 to queue4, store tasks with gradually increasing virtual runtimes. Subsequently, tasks are enqueued from queue0 to queue4 into a FIFO global queue, ensuring tasks with smaller virtual runtimes are placed at the front of the queue and thus scheduled first^①. If a task is a kernel thread, it is directly assigned to the CPU's local queue and allocated an infinite time slice to ensure timely execution^②.

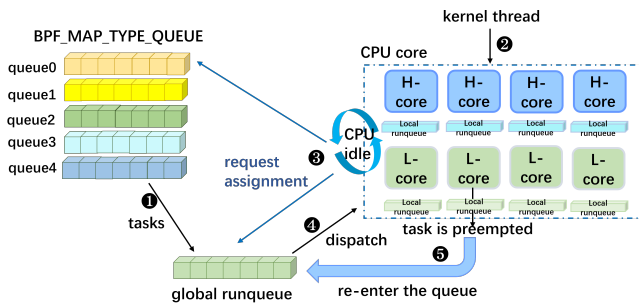


Fig. 2: SRAND scheduling policy logic.

We also monitor the state of each CPU, and when a CPU is idle and its local queue is empty, the CPU attempts to fetch tasks from the global queue. If the global queue is empty, the CPU requests tasks from the eBPF mapped queues, fetching tasks from queue0, which contains tasks with shorter virtual runtimes. Moreover, we perform affinity checks to select the

appropriate task for execution^③. After each task execution, we update the task's virtual runtime using the formula:

$$p_vtime+ = (slice - p_slice) \times \frac{100}{p_weight} \quad (1)$$

Here, tasks with higher weights have slower increments in virtual runtime. Additionally, we set a maximum timeout of 5000ms for tasks waiting to be scheduled to ensure timely responsiveness.

During task retrieval from the global queue by the local queue^④, core selection is also performed. Prior to each core selection, we gather task context information, mainly concerning the task's affinity, to determine whether the task can only run on a specific CPU or if the CPU on which the task previously ran is idle. If either of these conditions is met, the task is placed on the previous core it ran on. If the previous core is busy, we select an idle and affinity-compatible CPU from the remaining CPUs, allocate the task a default time slice of 20ms, wake up the CPU, and schedule the task on it. This process ensures selecting a suitable CPU for the task. If a task was preempted by a higher-priority task during its last execution, it is requeued to the global queue^⑤, where the appropriate CPU for task execution is determined by the global queue.

Subsequently, we implement the above strategy using the programmable scheduling framework `sched_ext`. `Sched_ext` introduces a new scheduling class called '`sched_ext_class`', which can be selected like most other classes via the `sched_setscheduler` method call. This is a non-privileged class, meaning any process can place itself into the `SCHED_EXT` class. Additionally, on systems utilizing the `sched_ext` framework, we can invoke the `scx_bpf_switch_all` method to migrate all processes running below real-time priority into the `SCHED_EXT` class. Building upon this, we load a custom BPF scheduling program in user space. This program takes over the scheduling responsibilities for all processes in the `SCHED_EXT` class, thus implementing the SRAND strategy proposed in this paper. The specific implementation framework is shown in the Fig.3.

In the user space, we implement a series of scheduling logic around the '`sched_ext_ops`'. Firstly, we overload the '`prep_enable`' hook, which is used to prepare tasks for enabling. We implement it to ensure the existence of task context data. If it's the first call for a task, meaning there's no data stored in memory, we attempt to create new storage using the '`BPF_LOCAL_STORAGE_GET_F_CREATE`' flag and allocate memory for the task. Simultaneously, we initialize the task's virtual runtime to the current system's virtual time.

Next, we implement the '`select_cpu`' function, responsible for core selection operations. Initially, it calls the '`bpf_task_storage_get`' function from the BPF program to retrieve task context data. It checks the task's affinity information and assigns the task to the CPU it previously ran

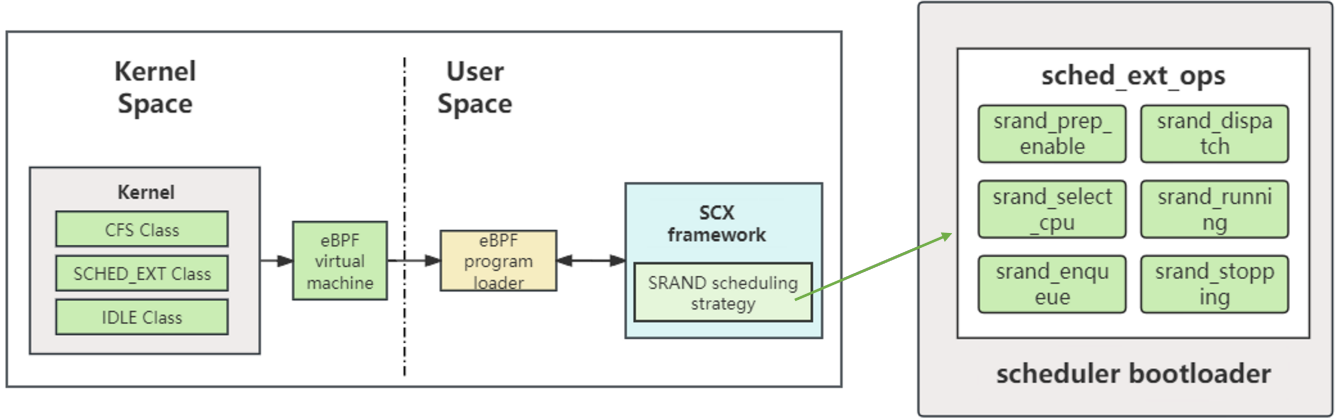


Fig. 3: Scheduling strategy method architecture based on scx.

on or uses the ‘scx_bpf_pick_idle_cpu’ function to select and wake up an idle CPU. Once a CPU is selected for a task, the ‘srand_enqueue’ function is invoked. This function first obtains the task’s context information and checks if the task is flagged as a kernel thread. If it’s a kernel thread, the ‘scx_bpf_dispatch’ method is called to directly place the kernel thread into the local queue.

For common tasks, we first retrieve the task’s virtual runtime and use the ‘vtime_before’ function to determine if the task’s virtual runtime is less than the system’s current virtual time minus a default time slice. If it is, we update the task’s virtual runtime to the system’s current virtual time minus the length of a default time slice. This design prevents tasks that have been asleep for a long time from having a too small virtual runtime, which could lead to longer individual task execution times and affect the execution of other tasks. Subsequently, we use the ‘bpf_map_push_elem’ function to temporarily store tasks with virtual runtimes less than 20ms, 50ms, 200ms, 500ms, and greater than or equal to 500ms in BPF mapped queues indexed from 0 to 4. These five queues are first-in-first-out, ensuring that the first-entered tasks in each eBPF queue are placed in the global queue first. If a task has been scheduled before and the enqueue flag ‘enq_flags’ is set to ‘SCX_ENQ_REENQ’, indicating that the task was requeued due to being preempted by a higher-priority scheduling class, the ‘scx_bpf_dispatch’ function is used to requeue the task to the global queue.

When a CPU requires a task, the balance_on function within the sched_ext framework is called. This function first checks if the CPU’s local queue has any tasks. If tasks are present, the pick_next_task_scx method is called to retrieve a task from the local queue directly onto the CPU for execution. If not, the consume_dispatch_q method is invoked to fetch tasks from the FIFO global queue and dispatch them to the local queue. If there are no tasks on the global queue, the srand_dispatch method of the SRAND strategy is triggered. This method first iterates over each CPU

using the bpf_for function and monitors signals emitted by CPUs with a dynamic array cpu_isneed_task. If a CPU’s local queue is empty, the value at the corresponding position of cpu_isneed_task becomes true, indicating that the CPU’s local queue has no tasks, and the CPU requires tasks for execution. At this point, the bpf_map_pop_elem method is called to retrieve tasks with smaller virtual runtimes from the BPF mapped queue queue0, placing them into the local queue of the CPU requiring tasks for execution. For other states, tasks are iteratively fetched from indexed BPF mapped queues from 0 to 4. The lower the index, the smaller the virtual runtime of tasks stored in the queue. Then, the scx_bpf_dispatch method is called to place tasks into the FIFO global queue, where task allocation is managed. This ensures that when tasks are selected and scheduled from the global queue, the tasks at the head of the queue are a batch with smaller virtual runtimes, prioritizing their execution. Additionally, each allocation from the FIFO global queue reduces the task selection time.

Afterwards, we overload the running and stopping hooks. When a task is running, the srand_running function is called to update the global virtual time. The vtime_before function checks if the system’s global virtual time is less than the current task’s virtual runtime. If it is, the global virtual time is updated to the task’s virtual runtime, ensuring the global virtual time always progresses forward. When a task stops running, the srand_stopping function is invoked to update the task’s virtual runtime. Finally, we set the maximum timeout for tasks waiting for scheduling in the sched_ext_ops structure as timeout_ms to 5000ms, ensuring timely task responsiveness. After implementing the scheduling logic, we use an eBPF program loader in the user space to inject the scheduling policy into the kernel, deploying the new strategy. By running scx_srand, the scheduling loading program is initiated, and tasks are orderly scheduled under our SRAND strategy.

IV. EXPERIMENTS

In this section, we evaluate the proposed scheduling strategy SRAND, and compare it with the original EEVDF scheduling strategy in Linux. We commence by testing the process context switching time within the scheduling policies. Subsequently, we employ the benchmark tool hackbench [14] to assess the performance and stress of the scheduling policies, drawing inspiration from prior work [15]. Additionally, we evaluate the load balancing performance of the two strategies to gauge their effectiveness.

These experiments were conducted on a computer featuring x86_64 architecture, equipped with Intel's 12th generation Core i5-12500H processor. The running environment comprises an Ubuntu 22.04 virtual machine equipped with 8 cores and 300GB of memory.

A. Process context delayed switching test

The time and frequency of context switching directly impact the overall system performance. Prolonged context switching times divert system resources from executing actual tasks, thereby diminishing system throughput and affecting task responsiveness and efficiency. To assess this, we utilize the Lmbench benchmark testing tool to measure the process context switching time of the two scheduling strategies.

Lmbench is a useful, portable micro-benchmark suite designed to measure important aspects of system performance [16]. Specifically, we employ Lmbench's [17] `lat_ctx` benchmark, which measures context switch times for different numbers and sizes of processes. These processes are interconnected in a Unix pipe ring, with each process reading the token from its pipe, performing the underlying work, and then passing the token to subsequent processes. Generally, the smaller the number of processes, the faster the context switch time.

To accurately reflect the performance of the scheduling strategy, we conduct tests on process sizes ranging from 0k to 64k and with process counts ranging from 2 to 15. Each test is executed 30 times, and the results are averaged. The tests are performed on a single CPU to mitigate interference from other communication times on the test results. The resulting test data is presented in Fig. 4.

According to the experimental results, we can intuitively feel from the Fig.4 regardless of the increasing number of processes or the size of processes, the SRAND scheduling strategy proposed in this paper achieves less context switch time compared to the generic Linux scheduling policy EEVDF. This reduction is attributed to SRAND's partitioning of tasks based on different virtual runtimes and the use of a FIFO global queue, which not only ensures tasks do not exceed their time limits and respond promptly but also reduces computational complexity. As a result, SRAND achieves an average reduction of 11.83% in context switch time compared to EEVDF, thereby improving system performance.

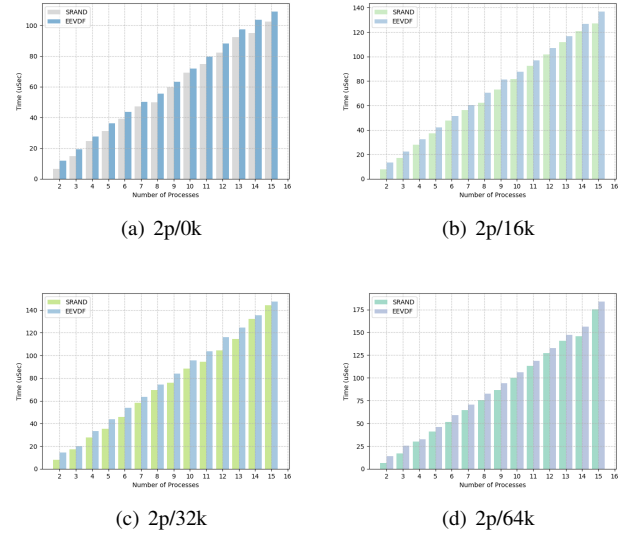


Fig. 4: Context switching time under different process sizes and numbers

B. Performance core stress test

Stress testing is employed to evaluate the performance of different scheduling policies when subjected to varying workloads. By imposing diverse loads on the system for short durations, we can effectively assess how different scheduling strategies manage task allocation under increasing pressure, particularly when the system generates numerous short-term tasks. To evaluate the performance of the scheduling policy proposed in this paper, we utilize the hackbench benchmarking tool to simulate frequent inter-process communication, thereby conducting comparative experiments between the SRAND and EEVDF scheduling policies.

Hackbench is part of a set of micro-benchmarks used by the creator of the Linux scheduler to compare its performance to other schedulers [18]. Its primary function involves creating a specified number of scheduling entity pairs (threads/processes), facilitating data transmission through sockets/pipe, and ultimately measuring the time overhead of the entire process. Lower time overhead indicates superior scheduling performance. In this paper, hackbench creates two groups: one acts as message senders, and the other as message receivers. Each sender sends a message to all receiving processes. We utilize sockets with a message size of 512 bytes, with each sender/receiver required to send/receive 200 messages. The number of initiated process groups ranges from 1 to 100, with increments of 5. Since each process group contains 40 processes, 40, 200, 400, 600, ..., 4000 processes are created. Each subprocess utilizes 25 file descriptors, and fork is employed to create sender and receiver processes.

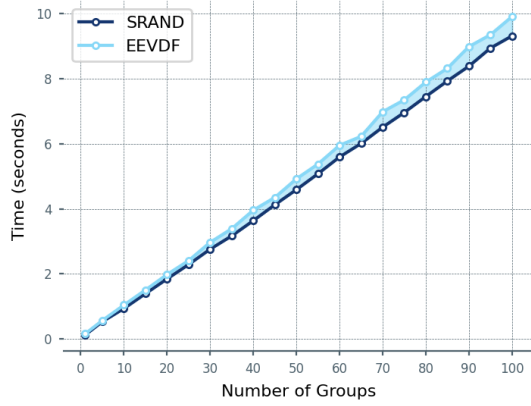


Fig. 5: Comparison of scheduling strategy performance test results.

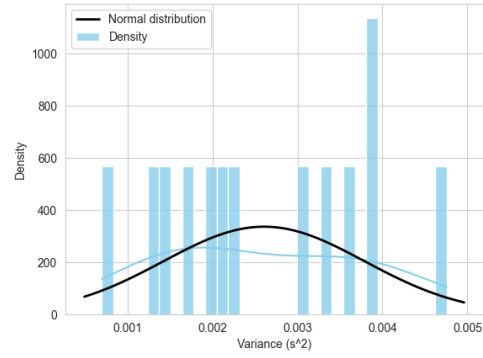
Through a series of experiments, we have obtained comparative results as depicted in Fig.5. From the graph, it is evident that under various workloads, the time overhead of the SRAND scheduling strategy consistently remains lower than that of EEVDF, indicating superior performance compared to the EEVDF scheduling policy. This improvement can be attributed to the more efficient management and allocation of tasks by the global queue in the scheduling strategy. On average, there is a reduction of 7.02% in time overhead.

C. Load balancing test

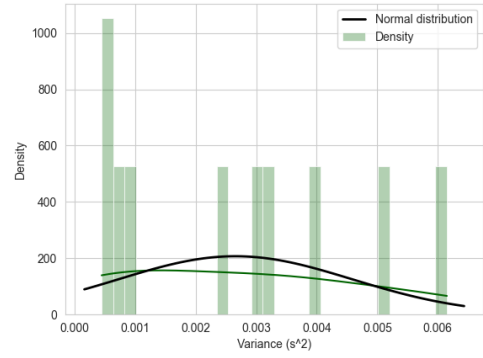
Load balancing refers to the reasonable allocation of tasks to various processors or computing nodes in the computing system to ensure that the load of each processor or computing node in the system is as balanced as possible, thereby improving the overall performance of the system. Load balancing can improve the quality of service (QoS) index [19]. A high QoS index means that the network operates stably, has good performance, and can provide the service quality expected by user.

We evaluate the performance of the two scheduling policies by utilizing CPU load balancing. While playing audio with QQ Music [20], we run both scheduling policies. Using the ‘trace_test_switchtime’ function to trace the CPU scheduling path, we collect the running time of each task on each CPU. This allows us to track the running and idle time of each CPU during this period. We focus solely on CPU running time and assess system load balancing based on the variance of CPU running time within each time interval. A variance closer to 0 indicates better load balancing.

Our experimental setup includes randomly selecting 10 timestamps, with each group consisting of 0.5 timestamps, resulting in 20 groups. The cumulative running time of all tasks on each CPU within each group constitutes the running time of that CPU. From this, we obtain a dataset of running



(a) SRAND



(b) EEVDF

Fig. 6: Load balancing of two scheduling strategies

time for each CPU group in each period, denoted as $X = x_1, x_2, \dots, x_8$, where x_i represents the running time of the i -th CPU in this cycle. We then calculated the average active time of the CPU in each group and calculated the variance of this time. Finally, we use kernel density estimation to visualize the variance data through density plots.

The experimental results depicted in Fig. 6 indicate that there is little disparity in load balancing between the two scheduling strategies. Under the SRAND policy, the difference in running time between CPUs remains within 0.005, while under the EEVDF policy, it stays within 0.006. Although there are slight differences, the better task allocation provided by the global queue in the SRAND policy results in superior load balancing compared to EEVDF.

V. CONCLUSION

The scheduler is a crucial component of an operating system [21], and effective scheduling strategies are essential for improving system performance. To address the issue of context switch overhead and performance degradation in desktop systems caused by a large number of short-term tasks or tasks requiring frequent communication, we

introduce a new scheduling policy called SRAND using the programmable scheduling framework sched_ext. It utilizes a five-level eBPF mapping queue to differentiate tasks based on their virtual runtime intervals, prioritizing tasks with smaller virtual runtimes by placing them in a FIFO-type global queue. During scheduling, tasks are selected from the head of the global queue, simplifying task selection while prioritizing tasks with smaller virtual runtimes to ensure timely execution. Additionally, SRAND considers task affinity and monitors CPU status, retrieving tasks from the eBPF mapping queue when needed and allocating them within the system. The scheduling policy is injected into the kernel using eBPF technology without the need for modifying kernel source code. Experimental comparisons demonstrate that our proposed SRAND scheduling policy achieves an average reduction of 11.83% in process context switch overhead compared to the Linux generic scheduling policy EEVDF. It also exhibits a 7.02% improvement in stress testing performance while maintaining relatively effective load balancing.

REFERENCES

- [1] Kaffes K, Humphries J T, Mazières D, et al. Syrup: User-defined scheduling across the stack[C]//Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. 2021: 605-620.
- [2] Pabla C S. Completely fair scheduler[J]. Linux Journal, 2009, 2009(184): 4.
- [3] Earliest Eligible Virtual Deadline First (EEVDF) scheduler, <https://lwn.net/Articles/925371/>
- [4] Stoica I, Abdel-Wahab H, Jeffay K, et al. A proportional share resource allocation algorithm for real-time, time-shared systems[C]//17th IEEE Real-Time Systems Symposium. IEEE, 1996: 288-299.
- [5] Abeni L, Lipari G, Buttazzo G. Constant bandwidth vs. proportional share resource allocation[C]//Proceedings IEEE International Conference on Multimedia Computing and Systems. IEEE, 1999, 2: 107-111.
- [6] Deck V S, Dempsey P. Theteardown: the gaming specialist has translated its open-hardware platform into a console[J]. Engineering & Technology, 2022, 17(4): 1-5.
- [7] Engler D R, Kaashoek M F, O'Toole Jr J. Exokernel: An operating system architecture for application-level resource management[J]. ACM SIGOPS Operating Systems Review, 1995, 29(5): 251-266.
- [8] Arpaci-Dusseau A C, Arpaci-Dusseau R H, Burnett N C, et al. Transforming policies into mechanisms with infokernel[J]. ACM SIGOPS Operating Systems Review, 2003, 37(5): 90-105.
- [9] Fried J, Ruan Z, Ousterhout A, et al. Caladan: Mitigating interference at microsecond timescales[C]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 2020: 281-297.
- [10] Humphries J T, Natu N, Chaugule A, et al. ghost: Fast & flexible user-space delegation of linux scheduling[C]//Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. 2021: 588-604.
- [11] Miano S, Risso F, Bernal M V, et al. A framework for eBPF-based network functions in an era of microservices[J]. IEEE Transactions on Network and Service Management, 2021, 18(1): 133-151.
- [12] Bertrone M, Miano S, Risso F, et al. Accelerating linux security with ebpf iptables[C]//Proceedings of the ACM SIGCOMM 2018 Conference on Posters and Demos. 2018: 108-110.
- [13] Ghigoff Y, Sopena J, Lazri K, et al. BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing[C]//18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21). 2021: 487-501.
- [14] Hackbench, <https://github.com/Ansen88/hackbenchv2.3>
- [15] Nabelsee M, Busse A, Parzyjegl H, et al. Load-aware scheduling for heterogeneous multi-core systems[C]//Proceedings of the 31st Annual ACM Symposium on Applied Computing. 2016: 1844-1851.
- [16] McVoy L W, Staelin C. Lmbench: Portable tools for performance analysis[C]//USENIX annual technical conference. 1996: 279-294.
- [17] Lmbench, <https://sourceforge.net/projects/lmbench/>
- [18] Anderson G, Marwala T, Nelwamondo F V. A response surface methodology approach to operating system scheduler tuning[C]//2010 IEEE International Conference on Systems, Man and Cybernetics. IEEE, 2010: 2684-2689.
- [19] Ghomi E J, Rahmani A M, Qader N N. Load-balancing algorithms in cloud computing: A survey[J]. Journal of Network and Computer Applications, 2017, 88: 50-71.
- [20] QQ Music, a music player, <https://y.qq.com/n/ryqq/>
- [21] Ma T, Chen S, Wu Y, et al. Efficient Scheduler Live Update for Linux Kernel with Modularization[C]//Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. 2023: 194-207.